ZEROTRACE

UNIFIED SYSTEM ARCHITECTURE & FILE HIERARCHY

Version 2.0

Author Aniruddh Gupta

Target Single Laptop (16GB RAM)

Year 2026

"They leave no trace. Because we found them first."

1. Executive Summary & Philosophy

ZeroTrace is an open-source, zero-cost, CPU-optimized cyber defense platform that combines three modular traces into a single source of truth:

- **Trace 1 (RECON):** Attack Surface Discovery — Subdomains, Ports, S3 buckets, SSL expiry, Shadow IT
- **Trace 2 (INTEL):** AI-Powered Vulnerability Prioritization — NVD + EPSS, XGBoost Exploit Model, Ollama Auto-Fix
- **Trace 3 (GUARD):** LLM Security Middleware Gateway — Regex pre-filter, ONNX DistilBERT classifier, PII scanner, SQLite audit log

> **Operational Readiness Score** = 100 − (0.40 × Asset Exposure + 0.40 × Vuln Urgency + 0.20 × AI Threat Exposure)

2. Complete File Hierarchy

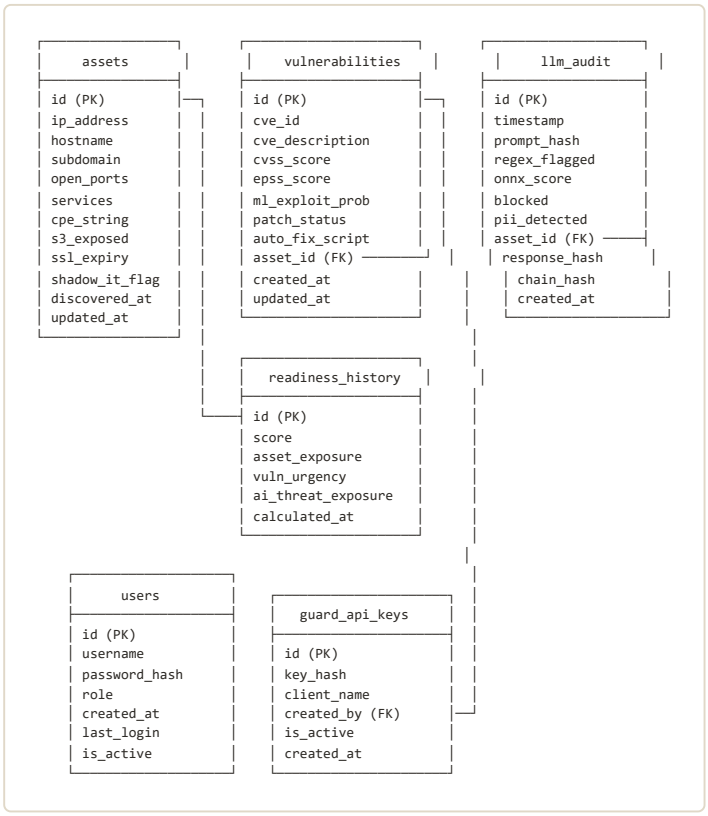
```
zerotrace/
├── docker-compose.yml              # Multi-container orchestration
├── Dockerfile.api                  # Docker build for FastAPI backend
├── Dockerfile.dashboard            # Docker build for Flask/HTMX dashboard
├── .env.example                    # Environment variable templates
├── .gitignore                      # Exclude bytecode, ONNX, SQLite, .env
├── Makefile                        # Dev commands (make run, test, migrate)
├── requirements.txt                # Python dependencies
├── README.md                       # Project setup and execution manual
│
├── config/                         # Configuration Management
│   ├── __init__.py
│   ├── settings.py                 # Pydantic BaseSettings from .env
│   ├── logging_config.py           # Structured JSON logging
│   └── database.py                 # SQLAlchemy engine, Redis client init
│
├── db/                             # Relational Data Layer
│   ├── alembic.ini                 # Migration configuration
│   ├── migrations/
│   │   ├── env.py
│   │   └── versions/
│   │       └── 0001_initial_schema.py
│   └── models/                     # SQLAlchemy ORM Models
│       ├── __init__.py
│       ├── asset.py                # IP, Hostname, Ports, Services, S3, SSL, Shadow IT
│       ├── vulnerability.py        # CVE, CVSS, EPSS, ML Exploit Score, Patch Status
│       ├── llm_audit.py            # Prompt hash, score, blocked flag, chain hash
│       ├── readiness.py            # Historical Readiness Score snapshots
│       └── user.py                 # 🆕 Username, bcrypt hash, role, is_active
│
├── modules/                        # Core Domain Logic (Three Traces)
│   ├── __init__.py
│   ├── recon/                      # TRACE 1: RECON
│   │   ├── __init__.py
│   │   ├── subdomains.py           # subfinder + cert-stream
│   │   ├── port_scanner.py         # Nmap top-100 port & banner wrapper
│   │   ├── s3_checker.py           # AWS S3 misconfiguration tester
│   │   ├── ssl_monitor.py          # SSL/TLS expiry & protocol checker
│   │   ├── shadow_it.py            # Inventory lookup & flagger
│   │   └── runner.py               # Orchestrator: async recon workflow
│   ├── intel/                      # TRACE 2: INTEL
│   │   ├── __init__.py
│   │   ├── nvd_feed.py             # NVD API ingestor
│   │   ├── epss_feed.py            # EPSS daily CSV parser
│   │   ├── cpe_matcher.py          # Service banner → CPE 2.3 → CVE
│   │   ├── ml_scorer.py            # XGBoost exploit probability
│   │   ├── patch_queue.py          # 48-hour prioritized queue
│   │   └── autofix_llm.py          # Ollama Qwen2.5-Coder script generator
│   └── guard/                      # TRACE 3: GUARD
│       ├── __init__.py
│       ├── proxy.py                # FastAPI reverse proxy /v1/chat/completions
│       ├── regex_filter.py         # Zero-latency heuristic & regex rules
│       ├── injection_classifier.py # ONNX DistilBERT semantic detector
│       ├── pii_scanner.py          # Outbound PII/API key scanner
│       ├── sqlite_logger.py        # Append-only hash-chained audit log
│       └── rate_limiter.py         # Redis token-bucket per client
│
├── core/                           # Unified System Core
```

```
│   ├── __init__.py
│   ├── readiness_calculator.py     # Weighted Readiness Score
│   ├── event_bus.py                # Redis Pub/Sub async dispatcher
│   └── redis_cache.py              # High-frequency query cache
│
├── auth/                           # 🆕 Authentication & Authorization
│   ├── __init__.py
│   ├── jwt_handler.py              # JWT creation, validation, refresh
│   ├── session_manager.py          # Flask-Login session management
│   ├── rbac.py                     # Role decorator (viewer, operator, admin)
│   └── password_utils.py           # Bcrypt hashing & verification
│
├── api/                            # Backend API Service (FastAPI)
│   ├── __init__.py
│   ├── main.py                     # Entrypoint, middleware, security headers
│   ├── routers/
│   │   ├── recon_router.py         # Trigger & query Recon scans
│   │   ├── intel_router.py         # CVE query, patch queue, auto-fix
│   │   ├── guard_router.py         # Proxy + Guard log endpoints
│   │   ├── readiness_router.py     # Readiness Score & metrics
│   │   └── auth_router.py          # 🆕 Login, logout, token, users
│   └── schemas/                    # Pydantic Request/Response
│       ├── recon_schema.py
│       ├── intel_schema.py
│       ├── guard_schema.py
│       ├── readiness_schema.py
│       └── auth_schema.py          # 🆕 LoginRequest, TokenResponse, UserCreate
│
├── dashboard/                      # Front-end UI
│   ├── app.py                      # Flask app + HTMX partials
│   ├── static/
│   │   ├── css/main.css            # Dark-mode glassmorphism styling
│   │   ├── js/app.js               # Alpine.js state + HTMX handlers
│   │   └── vendor/                 # Offline (no Node.js / npm)
│   │       ├── htmx.min.js         # HTMX 1.9.x (14KB)
│   │       └── alpine.min.js       # Alpine.js 3.x (15KB)
│   └── templates/
│       ├── base.html               # Layout, head, scripts, navbar
│       ├── login.html              # 🆕 Login page with CSRF token
│       ├── index.html              # Command Center Unified View
│       └── components/
│           ├── readiness_gauge.html    # SVG score ring (HTMX refresh)
│           ├── recon_panel.html        # Assets & open ports table
│           ├── intel_panel.html        # Patch queue & auto-fix trigger
│           ├── guard_panel.html        # Attack logs & block counters
│           └── autofix_modal.html      # Generated script display
│
├── models_weights/                 # 🆕 Downloaded at setup, NOT in Git
│   ├── distilbert_injection.onnx   # Quantized DistilBERT (~65MB)
│   ├── tokenizer_config.json       # HuggingFace tokenizer configs
│   └── xgboost_exploit_model.pkl   # Pre-trained XGBoost weights
│
├── scripts/                        # Utility & Setup
│   ├── download_models.py          # Fetch ONNX model & tokenizer
│   ├── seed_database.py            # Sample asset & threat data
│   ├── create_admin.py             # 🆕 Initial admin user creation
│   └── start_services.sh           # Docker entrypoint
│
└── tests/                          # Automated Test Suite (pytest)
    ├── conftest.py                 # Fixtures & database mock
    ├── test_recon.py               # Subdomain, nmap, S3, SSL tests
    ├── test_intel.py               # CPE, XGBoost, Ollama tests
```

```
├── test_guard.py              # Regex, ONNX, PII tests
├── test_readiness.py          # Score formula & monotonicity
├── test_auth.py               # 🆕 JWT, session, RBAC, password
└── test_integration.py        # 🆕 End-to-end pipeline tests
```

3. Database Schema NEW

3.1 Entity-Relationship Diagram



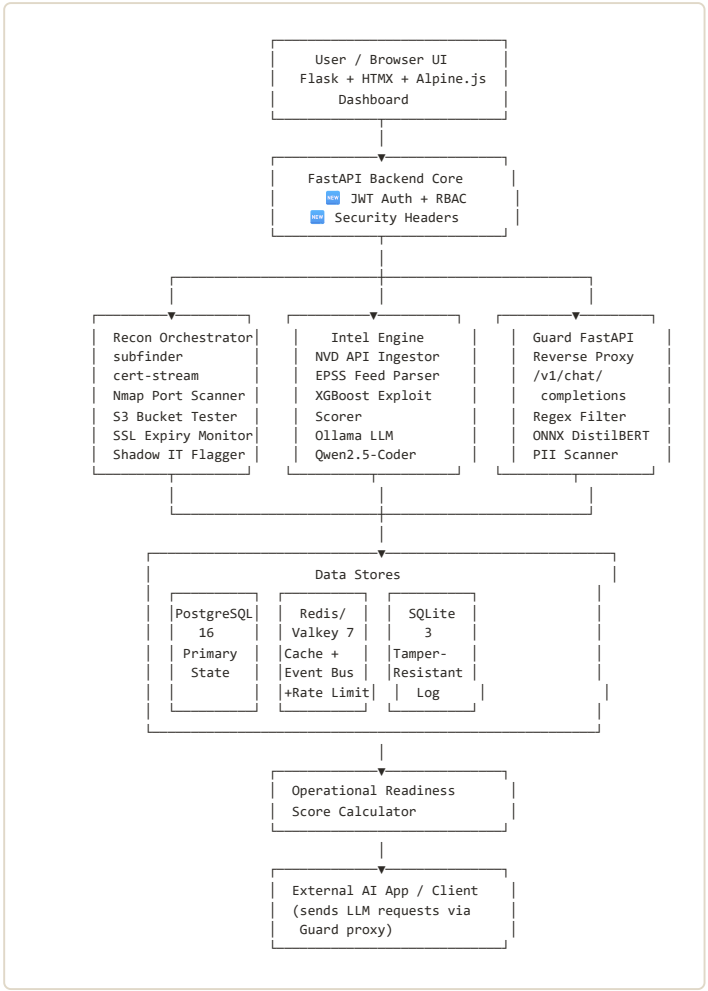
3.2 Key Relationships

RELATIONSHIP	TYPE	CONSTRAINT
Asset → Vulnerability	One-to-Many	One asset can have multiple CVEs; each CVE belongs to one asset
Asset → LLM Audit	One-to-Many	One asset can have multiple audit entries; each audit references one asset (nullable)
User → Guard API Keys	One-to-Many	One admin creates multiple API keys
Readiness History	Independent	Each row is a point-in-time snapshot; no FK required

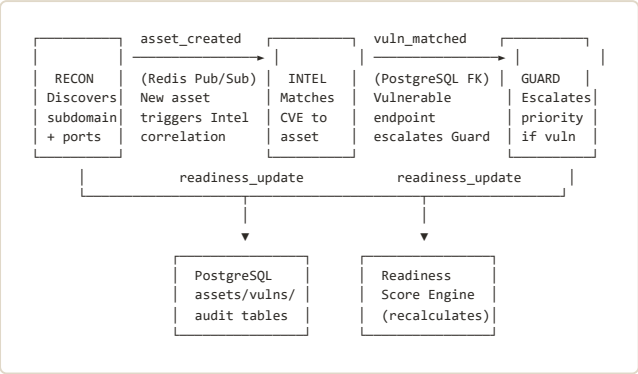
3.3 Indexes

TABLE	INDEX	PURPOSE
assets	idx_assets_ip	Fast lookup by IP address
assets	idx_assets_shadow_it	Fast filtering of Shadow IT assets
vulnerabilities	idx_vulns_asset_id	Fast join with assets table
vulnerabilities	idx_vulns_cve_id	Fast CVE lookup
vulnerabilities	idx_vulns_patch_status	Fast filtering of unpatched vulns
llm_audit	idx_audit_timestamp	Fast time-range queries
llm_audit	idx_audit_blocked	Fast filtering of blocked requests
readiness_history	idx_readiness_at	Fast time-series queries

4. Master Architecture Topology

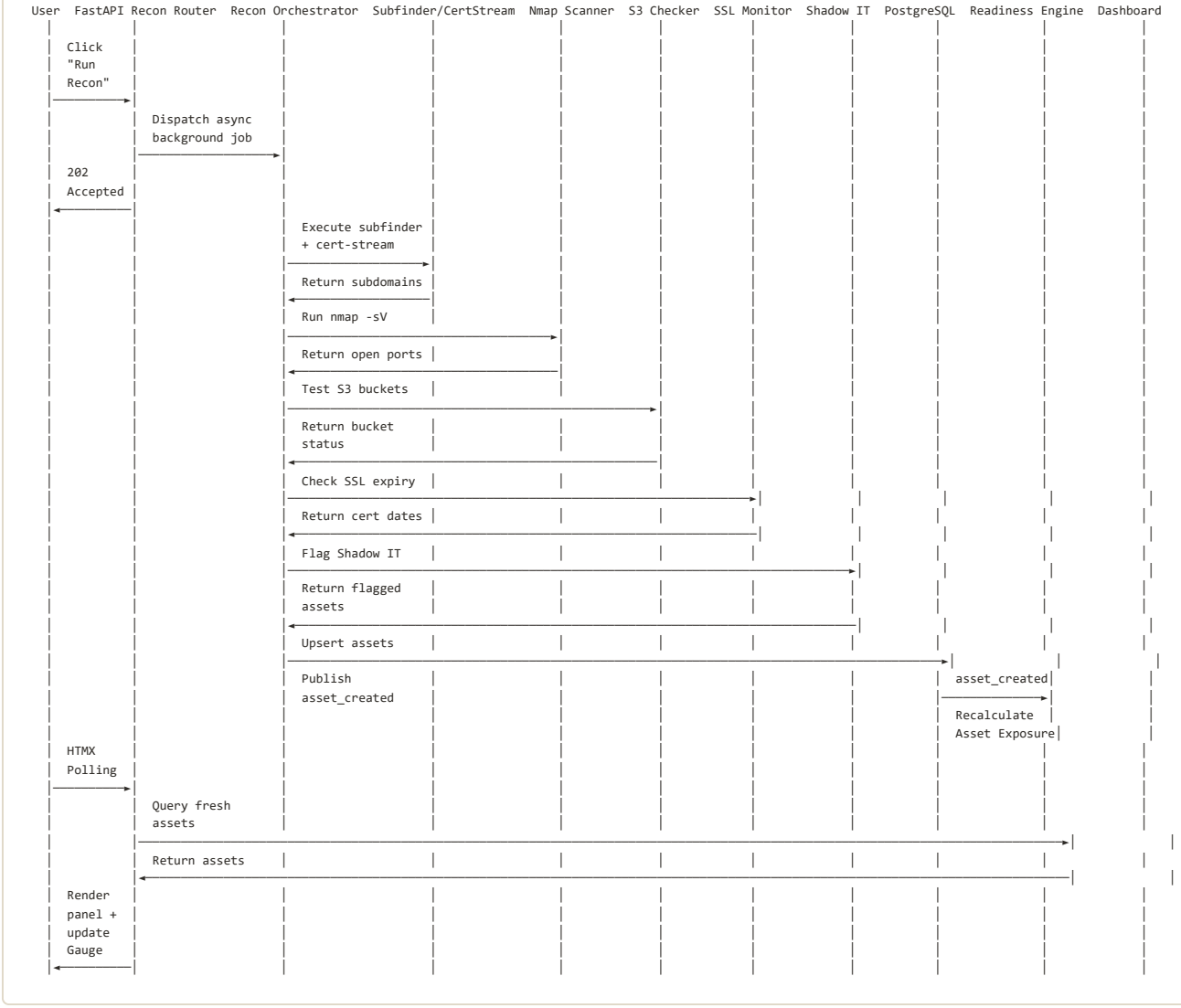


5. Cross-Module Event Flow NEW

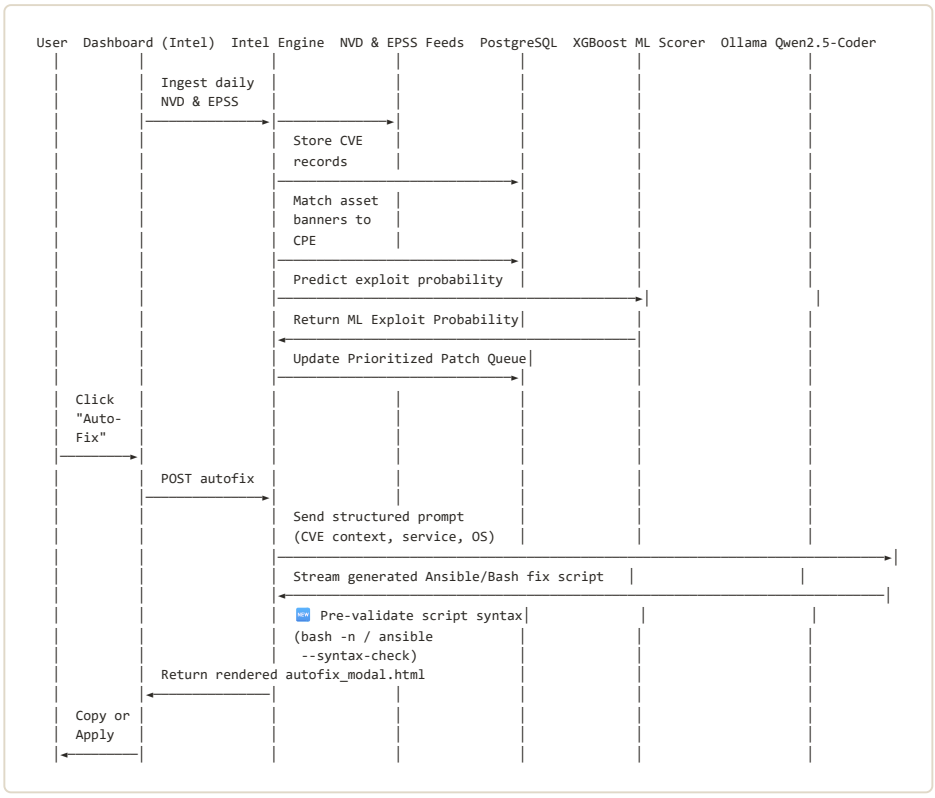


EVENT	CHANNEL	PUBLISHER	SUBSCRIBER	ACTION
`asset_created`	Redis `zt:events`	Recon runner	Intel engine	Auto-correlate CVEs for new asset
`vuln_matched`	Redis `zt:events`	Intel engine	Guard proxy	Escalate priority for vulnerable endpoints
`attack_blocked`	Redis `zt:events`	Guard proxy	Readiness engine	Recalculate AI Threat Exposure
`vuln_patched`	Redis `zt:events`	Intel auto-fix	Recon runner	Re-scan to verify patch
`readiness_update`	Redis `zt:events`	Readiness engine	Dashboard	HTMX push to readiness gauge

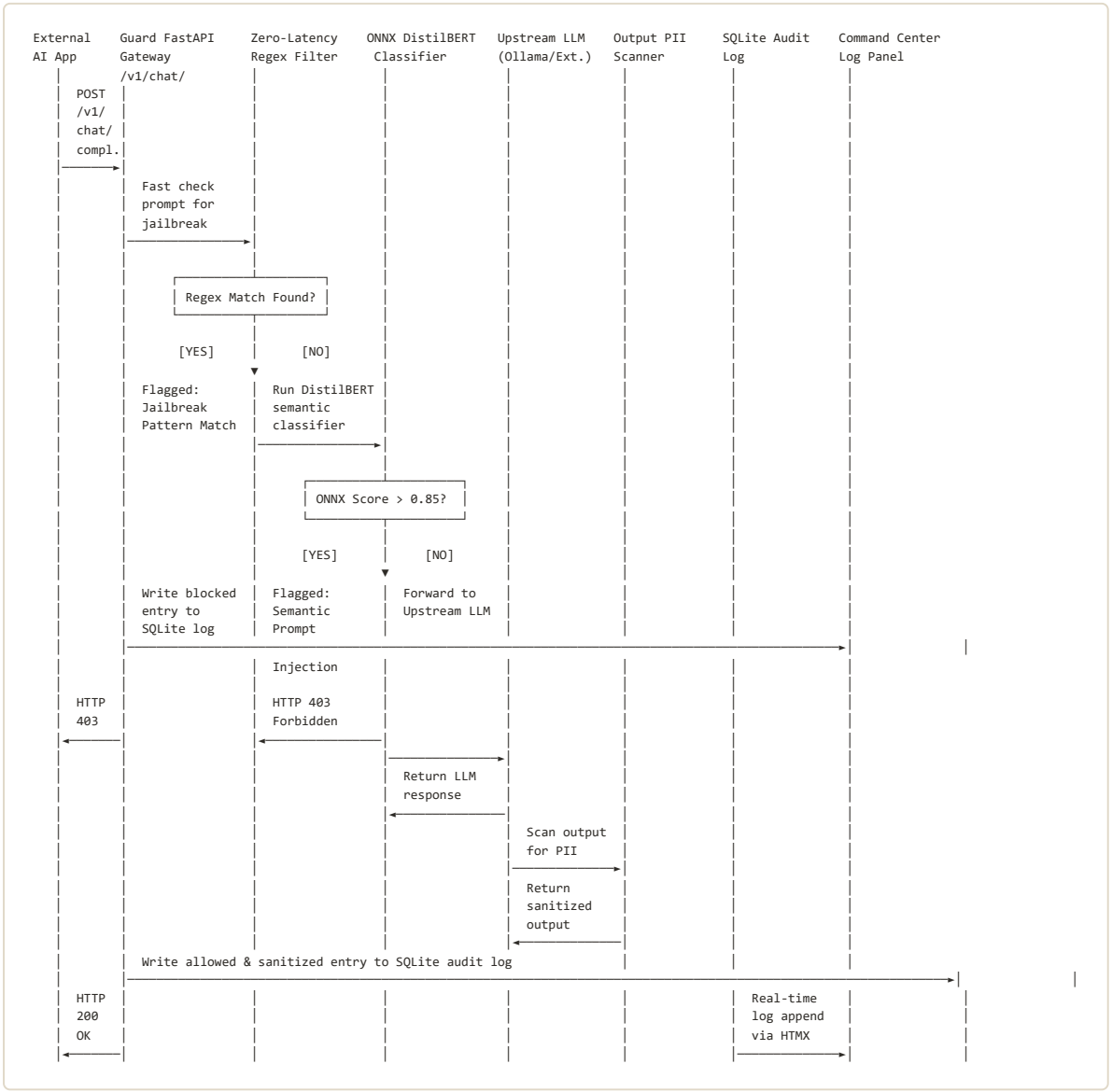
6. Trace 1: RECON Click-to-Output Flow



7. Trace 2: INTEL Vulnerability & Auto-Fix Flow



8. Trace 3: GUARD LLM Gateway Interception Flow



9. Memory Footprint Allocation

COMPONENT / SERVICE	RAM USAGE	STATUS / OPTIMIZATION
Windows 11 OS	~4.0 GB	Background OS
WSL 2 (Ubuntu + Docker)	~2.0 GB	Linux Subsystem
PostgreSQL 16	~500 MB	Primary Relational DB
Redis (Valkey 7)	~300 MB	Cache, Rate Limiter, Event Bus
FastAPI Backend	~600 MB	Async API Engine
Ollama (Qwen2.5-Coder 7B Q4_K_M)	~4.5 GB	Quantized Local LLM
ONNX Runtime (DistilBERT)	~400 MB	CPU Inference Engine
Flask Dashboard	~200 MB	Presentation Tier
Total Active Memory	**~9.0 GB**	**Comfortable on 16GB–20GB RAM**

10. API Specification NEW

10.1 Recon Router (`/api/v1/recon`)

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/scan`	Operator+	Trigger recon scan for a domain
GET	`/results`	Viewer+	Get all discovered assets
GET	`/results/{asset_id}`	Viewer+	Get single asset details
GET	`/shadow-it`	Viewer+	Get all Shadow IT flagged assets
DELETE	`/results/{asset_id}`	Admin	Remove asset from inventory

10.2 Intel Router (`/api/v1/intel`)

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	`/cves`	Viewer+	Get prioritized CVE list
GET	`/cves/{cve_id}`	Viewer+	Get single CVE details
GET	`/patch-queue`	Viewer+	Get 48-hour prioritized patch queue
POST	`/autofix`	Operator+	Generate auto-fix script for a CVE
PATCH	`/cves/{cve_id}/status`	Operator+	Update patch status (pending → patched)

10.3 Guard Router (`/api/v1/guard`)

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/v1/chat/completions`	API Key	Proxy endpoint (intercept LLM requests)
GET	`/logs`	Viewer+	Get audit log entries
GET	`/stats`	Viewer+	Get blocked/allowed counts
GET	`/health`	None	Health check endpoint

10.4 Readiness Router (`/api/v1/readiness`)

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	`/score`	Viewer+	Get current Operational Readiness Score
GET	`/history`	Viewer+	Get score history (time series)
GET	`/breakdown`	Viewer+	Get score breakdown by component

10.5 Auth Router (`/api/v1/auth`) NEW

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/login`	None	Authenticate and receive JWT
POST	`/refresh`	JWT	Refresh expiring JWT
POST	`/logout`	JWT	Invalidate session
GET	`/me`	JWT	Get current user info
POST	`/users`	Admin	Create new user
PATCH	`/users/{user_id}`	Admin	Update user role

11. Docker Compose Service Map NEW

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: zerotrace
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - internal
    # No port exposed to host — internal only

  redis:
    image: valkey/valkey:7
    networks:
      - internal

  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    ports:
      - "8000:8000"
    depends_on:
      - postgres
      - redis
    networks:
      - internal
      - external
    environment:
      - DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/zerotrace
      - REDIS_URL=redis://redis:6379
      - JWT_SECRET=${JWT_SECRET}

  dashboard:
    build:
      context: .
      dockerfile: Dockerfile.dashboard
    ports:
      - "5000:5000"
    depends_on:
      - api
    networks:
      - internal
      - external

  ollama:
    image: ollama/ollama:latest
    volumes:
      - ollama_data:/root/.ollama
    networks:
      - internal

volumes:
  pgdata:
```

```
  ollama_data:

networks:
  internal:
    driver: bridge
  external:
    driver: bridge
```

Key Security Decision: PostgreSQL and Redis are on `internal` network only — no ports exposed to the host machine. Only `api` (port 8000) and `dashboard` (port 5000) are accessible externally.

END OF ARCHITECTURE

> *"They leave no trace. Because we found them first."*

ZeroTrace Project — Version 2.0 — 2026
Prepared by Aniruddh Gupta